

Escuela Técnica N°1 Otto Krause



Laboratorio de Programación Orientada a Objetos

Guión - Ejercitación 14

**Alumnos: Toledano Jazmin
Pagni Octavio**

Curso: 5°2 Computación

Prof: Aaron Sebastian Serrano

Ciclo Lectivo 2026

Consignas:

1. Idea del juego en 3-4 líneas: de qué se trata, qué hace el jugador.

Decidimos utilizar la idea de nuestro proyecto cuatrimestral de proyecto informático, un juego de terror psicológico ambientado en el buffet de la escuela.

En este juego, el jugador tiene que hacer tareas y recorrer el lugar. Mientras avanza aparecen anomalías que tiene que detectar, y termina descubriendo un secreto del buffet.

ORDEN DE LAS CARPETAS Y SUS RESPECTIVAS CLASES:

MODEL

- |
- |— Entidad
- |— Personaje
- |— Jugador
- |— NPC
- |— Objeto
- |— Anomalía
- |— Tarea
- |— Inventario
- |— Zona

INTERFACES

- |
- |— Interactable

DAO

- |
- |— EntidadDAO
- |— Implementación correspondiente

SERVICE

- |
- |— EntidadService

EXCEPCIONES

- |
- |— AnomaliaDetectadaException
- |— InteraccionInvalidaException
- |— TareaNoDisponibleException
- |— ObjetoNoDisponibleException
- |— DiaInvalidoException

MAIN

- |

└─ Main

2. Las clases del modelo: cuál es la clase base, cuáles heredan, y por qué eligieron esa jerarquía (qué comparten, qué las diferencia).

La clase base de nuestro modelo es **Mundo**, que será una clase abstracta. Esta clase tendrá los atributos que comparten los elementos principales del juego, como **id**, **nombre** y **posición**.

Mundo heredan 3 subclases, que son las siguientes:

Personaje

↓

Objeto

↓

Anomalía

Personaje hereda a Jugador y NPCs

Objeto no hereda a nada, ya que representa los objetos que pueden encontrar y utilizarse.

Anomalía lo mismo, ya que representa los eventos o elementos extraños que aparecen dentro del juego.

Existen también las clases, **Tarea**, **Inventario** y **Zona**, que se encargan de representar las tareas del jugador, los objetos que posee y las distintas zonas del buffet.

Elegimos esta jerarquía porque todas estas clases forman parte del mismo **Mundo** de juego e incluso comparten las 3 características principales, pero independientemente tienen sus propias características y comportamientos. Por ejemplo, un **Jugador** tiene el día actual, estado, cordura y tareas completadas, mientras que una **Anomalía** tiene nivel de peligro, descripción, día de aparición y si fue detectada.

3. Atributos de cada clase (los del padre + los específicos de cada hija).

Atributos de las clases:

Mundo

- id
- nombre
- posición

Personaje

Además de los atributos heredados de Entidad:

- estado

Jugador

Además de los atributos heredados de Personaje:

- diaActual
- cordura
- tareasCompletadas

NPC

Además de los atributos heredados de Personaje:

- rol
- diálogo
- puedeDarTarea

Objeto

Además de los atributos heredados de Entidad:

- tipoObjeto
- utilizable
- estado

Anomalía

Además de los atributos heredados de Entidad:

- nivelPeligro
- descripción
- diaAparición
- detectada

Tarea

La clase Tarea representará las actividades que el jugador debe completar durante los diferentes días.

- id
- descripción
- dia
- completada

Inventario

La clase Inventario permitirá almacenar los objetos que el jugador posee durante la partida.

- objetos

Zona

La clase Zona representará las diferentes zonas que forman parte del buffet y que el jugador puede recorrer.

- id
- nombre
- descripción

4. Diseño de la tabla en MySQL (el CREATE TABLE, igual que venimos haciendo, con la columna tipo si van a usar tabla única para las subclases).

Decidimos utilizar una única tabla, como venimos haciendo desde que empezamos este tema para que sea más simple.

```
CREATE TABLE entidades (
  id INT PRIMARY KEY AUTO_INCREMENT,
  nombre VARCHAR(50) NOT NULL,
  posicion VARCHAR(50) NOT NULL,
  tipo VARCHAR(30) NOT NULL,

  estado VARCHAR(50),

  dia_actual INT,
  cordura INT,
  tareas_completadas INT,

  rol VARCHAR(50),
  dialogo VARCHAR(255),
  puede_dar_tarea BOOLEAN,

  tipo_objeto VARCHAR(50),
  utilizable BOOLEAN,

  nivel_peligro INT,
  descripcion VARCHAR(255),
  dia_aparicion INT,
  detectada BOOLEAN
);
```

El "**tipo**" sirve para saber si es un jugador, empleado, objeto o anomalía.

5. Lista de reglas de negocio que va a validar el service (mínimo 5), y qué excepción dispara cada una.

1. Si encuentra una anomalía: AnomaliaDetectadaException
2. Si no se puede interactuar: InteraccionInvalidaException
3. Si la tarea no está disponible: TareaNoDisponibleException
5. Si el objeto no está disponible: ObjetoNoDisponibleException
6. Si el día está mal: DiaInvalidoException

6. Qué interfaz van a definir y en qué método del service la van a usar de forma polimórfica.

Interfaz que vamos a usar: **Interactuable**

Método en que la vamos a usar: **void interactuar();**

La interfaz va a ser usada en clases que puedan tener interacción. (**Ej: el objeto**)

7. Métodos previstos del service (nombres, no implementación)

Métodos para las entidades

crearEntidad()
buscarEntidad()
listarEntidades()
eliminarEntidad()

Métodos para la jugabilidad

realizarTarea()
usarObjeto()
detectarAnomalia()
interactuar()
avanzarDia()

Método polimórfico

interactuar(Interactuable elemento)

El Service podrá recibir diferentes tipos de objetos interactuables y ejecutar su comportamiento correspondiente sin depender de una subclase específica.

Fin:D